

Intro to InfoSec & Linux Fundamentals

Systems Architecture, General Linux Command Arsenal, and Lab Setup

Instructor: Yahya Zaghloul

Course: Cybersecurity Foundations

// 1. Information Security Foundations

Cybersecurity is the discipline of protecting digital networks, hosts, applications, and sensitive information from unauthorized intrusion, disruption, or alteration. Security is fundamentally an engineering exercise in risk management rather than seeking an unattainable state of absolute invulnerability.

The CIA Triad

- **Confidentiality:** Guarantees that sensitive data is accessible solely to authorized users and processes. Enforced via encryption (AES, RSA), strict access controls, and least-privilege policies.
- **Integrity:** Guarantees that information and software have not been maliciously or accidentally altered. Enforced via cryptographic hash functions (SHA-256) and digital signatures.
- **Availability:** Ensures authorized entities have reliable, timely access to services and systems. Preserved via architectural redundancy, load balancing, and anti-DDoS mitigations.

The Risk Model & Core Terminology

SECURITY EQUATION

$\text{Risk} = \text{Threat Actor} \times \text{Vulnerability} \times \text{Impact}$. Security mitigations actively minimize vulnerabilities and contain potential business impact.

- **Vulnerability:** A weakness or flaw in system architecture, implementation code, or configuration (e.g., buffer overflow, unpatched service, default password).
- **Threat Actor:** Any adversary (malicious hacker, insider, automated worm) possessing the motive and capability to target a vulnerability.
- **Exploit:** The tailored software mechanism, payload, or technique used to capitalize on a vulnerability to trigger unauthorized actions.

// 2. Operating Systems & Linux Architecture

An Operating System (OS) is the software layer mediating hardware resources and application requests.

- **Hardware Layer:** Physical CPU (x86_64/ARM), RAM, storage disks (NVMe/SSD), and Network Interface Cards (NICs).
- **Kernel Space (Ring 0):** The privileged core of the OS. Directly governs CPU process scheduling, virtual memory management, hardware device drivers, and network sockets.
- **System Calls (Syscalls):** The guarded API boundary through which applications request kernel services (e.g., open, read, write, socket, fork).
- **User Space (Ring 3):** Isolated unprivileged space where standard user programs execute (e.g., Bash, Python, web browsers). Crashes here never destabilize the kernel.
- **The Shell:** A command-line interpreter (Bash, Zsh) that reads user input, resolves paths, requests system calls, and executes child processes.

// 3. Linux Filesystem Hierarchy Standard (FHS)

In Linux, 'everything is a file' -- disks, network sockets, active processes, and terminals are represented directly within the filesystem tree rooted at /.

- **/ (Root):** The top-level root directory containing all subtrees, devices, and mount points.
- **/bin & /usr/bin:** Standard user command binaries required for system operation (ls, cat, grep, bash).
- **/sbin & /usr/sbin:** System-level administrative binaries reserved for the root superuser (iptables, fdisk).
- **/etc:** System-wide host configuration files (passwd, shadow, network configs, sudoers).
- **/home:** Personal user directories and workspace files (e.g., /home/kali).
- **/root:** The dedicated home directory for the root superuser.
- **/var/log:** Dynamic variable runtime data, authentication audits, and system service logs.
- **/tmp:** Temporary scratchpad directory; world-writable by default across all accounts.
- **/dev:** Device nodes mapping physical disks (/dev/sda) and pseudo-devices (/dev/null, /dev/urandom).
- **/proc:** Virtual pseudo-filesystem reflecting live kernel memory and active process metrics (PIDs).

// 4. Permissions & Security Architecture

Linux permissions regulate access across three distinct tiers: Owner (u), Group (g), and Others/World (o).

```
// permissions_anatomy.txt
- rwx r-x r-- 1 root root 4096 Sep 12 10:00 secret.txt
# Breakdown:
# [-] : File type (- = regular file, d = directory, l = symbolic link)
# [rwx] : Owner permissions (Read = 4, Write = 2, Execute = 1 -> Sum = 7)
# [r-x] : Group permissions (Read = 4, Execute = 1 -> Sum = 5)
# [r--] : Others/World permissions (Read = 4 -> Sum = 4)
```

Standard permission modes include 755 (standard executable/script) and 600 (confidential file, readable and writable only by the owner).

```
// chmod_chown_examples.sh
$ chmod 600 id_rsa # Restrict file to owner read/write only
$ chmod 755 run_tool.sh # Make script executable by all, editable by owner
$ chown kali:kali file.txt # Change user and group ownership
```

// 5. The Golden Rule: How to Help Yourself

INSTRUCTOR AXIOM

Do not memorize commands or flags. Develop the skill to interrogate the system and locate whatever parameter you need within 10 seconds.

```
// self_research_toolkit.sh
# 1. Read the complete official manual for any command:
$ man find # In man: press '/' to search, 'n' for next match, 'q' to quit

# 2. Fast flag summary:
$ ls --help # Prints quick options summary

# 3. What does this command do? (concise description):
$ whatis chmod # One-line manual description

# 4. Search manuals by concept or keyword:
$ apropos "routing" # Finds all command manuals matching keyword
```

// 6. General Linux Command Arsenal by Category

Mastering the Linux terminal requires internalizing commands by functional category rather than ad-hoc memorization.

Category 1: Navigation & System Information

```
// navigation_and_system.sh
$ whoami           # Prints the current active username
$ id              # Displays current UID, GID, and assigned group memberships
$ uname -a        # Displays complete Linux kernel version and CPU architecture
$ hostname        # Displays system network hostname
$ pwd            # Print Working Directory (current absolute path)
$ ls -la         # List all files including hidden dotfiles and permission flags
$ ls -lh         # List files with human-readable sizes (KB, MB, GB)
$ cd /path/to/dir # Change directory (cd .. = up one level, cd ~ = home)
$ history        # View chronological list of previously typed commands
$ clear          # Reset terminal screen buffer
```

Category 2: File & Directory Manipulation

```
// file_manipulation.sh
$ touch filename    # Create empty file or update timestamps
$ mkdir dirname    # Create a new directory
$ mkdir -p a/b/c    # Create nested parent directories recursively
$ cp src dst       # Copy a file
$ cp -r dir1 dir2   # Copy a directory recursively
$ mv old new        # Move or rename files and directories
$ rm filename       # Delete a file
$ rm -rf dirname    # Force recursive deletion of directory (Caution!)
$ ln -s target link # Create a symbolic link (shortcut)

# Handling Special Filenames:
$ cat ./-filename   # Escape files starting with '-' using relative path prefix
$ cat "file with space" # Escape filenames with spaces using double quotes
$ cat file\ with\ space # Escape filenames with spaces using backslashes
```

Category 3: Reading & Inspecting File Content

```
// content_inspection.sh
$ cat filename      # Concatenate and display full file contents
$ less filename     # Paginated interactive viewer (ideal for large logs; q to quit)
$ head -n 20 file   # Print the first 20 lines of a file
$ tail -n 20 file   # Print the last 20 lines of a file
$ tail -f logfile    # Stream and follow updates added to file in real-time
$ wc -l file        # Count total lines in file
$ wc -c file        # Count total bytes in file
$ file filename     # Inspect true MIME type via Magic Bytes (ignores file extensions)
```

Category 4: Searching & Locating Assets

```
// asset_discovery.sh
# The 'find' utility traverses directory trees recursively based on criteria:
# Syntax: find [search_path] [filters]
$ find . -name "*.conf"           # Find files by name (case-sensitive)
$ find . -iname "readme"         # Find files by name (case-insensitive)
$ find . -type f                 # Match regular files only (ignores directories)
$ find . -type d                 # Match directories only
$ find . -size 1033c             # Match files of exact size (c=bytes, k=KB, M=MB)
$ find . -user username         # Match files owned by specific user
$ find . -group groupname       # Match files owned by specific group
$ find . ! -executable          # Negate condition (files that are NOT executable)

# Executable Binary Lookup:
$ which python3                 # Print exact executable binary path from $PATH
$ whereis bash                  # Print binary, man page, and source locations
$ locate filename               # Fast search using pre-indexed system database
```

Category 5: Standard Streams & I/O Redirection

Every Linux process inherits three standard POSIX I/O channels:

- **FD 0 (stdin):** Standard Input (keyboard input or piped stream).
- **FD 1 (stdout):** Standard Output (normal terminal display stream).
- **FD 2 (stderr):** Standard Error (error diagnostics stream).

```
// io_redirection.sh
$ command > file.txt             # Redirect stdout to file (overwrites existing content)
$ command >> file.txt            # Redirect stdout to file (appends to end of file)
$ command < file.txt            # Feed file content as stdin to command
$ command 2> /dev/null          # Discard error messages (stderr) into the null black hole
$ command > out.log 2>&1        # Redirect both stdout and stderr into same log file

# Real-world usage: search system tree without 'Permission denied' noise:
$ find / -type f -name "*.key" 2>/dev/null
```

Category 6: Pipelines & Text Processing

The Unix pipe (|) connects the stdout of one program directly into the stdin of another, allowing you to assemble custom data analysis pipelines on the fly.

```
// pipelines_and_text.sh
# Searching text with grep:
$ grep "admin" /etc/passwd          # Search for pattern inside file
$ grep -i "token" file.txt          # Case-insensitive pattern match
$ grep -n "error" app.log           # Include line numbers of matching lines
$ grep -v "debug" app.log           # Invert match (show lines that do NOT match)

# Sorting and Unique line extraction:
$ sort data.txt                     # Sort lines alphabetically or numerically
$ sort data.txt | uniq -u            # Output only lines that appear strictly ONCE
# CRITICAL: uniq only compares adjacent lines; input MUST be sorted first!

# Delimited column extraction with cut:
$ cut -d ":" -f 1 /etc/passwd       # Extract usernames from colon-separated file
```

Category 7: Data Extraction, Archives & Networking

```
// data_and_networking.sh
# Extract printable ASCII characters from binary files or memory dumps:
$ strings binary_file | grep "==="

# Base64 Encoding / Decoding:
$ base64 file.txt                   # Encode file to Base64 ASCII
$ base64 -d encoded.txt             # Decode Base64 data (Base64 is NOT encryption!)

# Compressed Archives:
$ tar -czvf backup.tar.gz /folder  # Create compressed gzip tarball
$ tar -xzvf archive.tar.gz          # Extract compressed gzip tarball
$ zip -r archive.zip /folder        # Create zip archive
$ unzip archive.zip                 # Extract zip archive

# Remote Access & Basic Networking:
$ ssh -p 2220 user@hostname         # Connect via SSH on custom port (default is 22)
$ ping -c 3 8.8.8.8                 # Test network host reachability
$ curl -I https://example.com       # Inspect HTTP response headers
```

// 7. Practical Assignment: OverTheWire Bandit (0-10)

THE MISSION

Connect to OverTheWire Bandit and conquer Levels 0 through 10 before Session 2.
Apply the general command categories learned today and use man pages to research solutions independently.

```
// bandit_connection.sh
# Level 0 Connection Coordinates:
$ ssh -p 2220 bandit0@bandit.labs.overthewire.org
# Password: bandit0
```

Rules of Engagement:

- **Zero Spoilers:** Do not search for writeups or flag walkthroughs online. True tradecraft is forged when you struggle through syntax and manual pages.
- **Lab Notebook:** Maintain an encrypted lab notebook. Record: (1) Username, (2) Found Password, and (3) The exact command sequence used.

// 8. Complete Lab Setup: VMware & Kali Linux

A virtual machine (VM) provides a sandboxed security laboratory. It ensures that testing exploits and tools never compromises your primary host operating system.

STEP 01 Download & Install VMware Workstation

Download VMware Workstation Pro (officially free for personal use) or VMware Workstation Player for your host operating system (Windows or Linux).

Official Download:

<https://www.vmware.com/products/desktop-hypervisors/workstation-and-fusion>

Run the installer using default configuration options. Reboot host PC if prompted by the setup wizard.

STEP 02 Download Pre-Built Kali Linux VM Image

Navigate to the official Kali download page:

<https://www.kali.org/get-kali/#kali-virtual-machines>

Select the 'VMware (64-bit)' pre-built image option and download the 7-Zip archive (.7z). Do NOT download the generic ISO installer; the pre-built image includes pre-configured VMware Tools, drivers, and display settings.

STEP 03 Extract the Archive Using 7-Zip

Install 7-Zip from <https://www.7-zip.org> if not already installed on your host system.

Right-click the downloaded .7z file, select 7-Zip -> Extract to

'kali-linux-vmware...'. Ensure you extract to a drive partition with at least 35 GB of free storage (preferably an SSD).

STEP 04 Import and Launch the VM in VMware

Open VMware Workstation. Click 'File' -> 'Open...' (or 'Open a Virtual Machine' on the welcome screen).

Navigate into the extracted folder and select the virtual machine configuration file ending with the .vmx extension (e.g., kali-linux-202X.X-vmware-amd64.vmx).

Click 'Power on this virtual machine'. When prompted with 'Did you move or copy this virtual machine?', ALWAYS select 'I Copied It'.

STEP 05 Default Login & Post-Install Verification

At the graphical login prompt, enter: Username: 'kali' | Password: 'kali'

Open the terminal (Ctrl + Alt + T) and verify networking by running: `ping -c 3 8.8.8.8`

Update the package repository lists: `sudo apt update` (password: kali)

Troubleshooting Virtualization Issues

- **VT-x / AMD-V BIOS Error:** If VMware displays an error stating hardware virtualization is disabled, restart your PC, enter BIOS/UEFI settings (usually F2, F10, or Del), and enable 'Intel VT-x' or 'AMD SVM Mode'.
- **Releasing Mouse Cursor:** Press Ctrl + Alt simultaneously to release mouse focus from the VM window back to your host desktop.
- **Network Connectivity:** Ensure VM Network Adapter is set to 'NAT' mode under VM Settings -> Network Adapter.